Try this tonight: draw an FSM for a real-world gadget

Example: A power bank with one button and 4 LEDs.

Step-by-step

1. List states: OFF, ON_IDLE, CHARGING_PHONE, LOW_BATTERY.
2. List inputs: button press, USB load detected, battery low.
3. Draw transitions:
   - OFF → ON_IDLE when button pressed
   - ON_IDLE → CHARGING_PHONE when load detected
   - Any → LOW_BATTERY when battery low
4. Define outputs per state: which LEDs light up, whether boost converter enables.
5. Congratulations: you've described a product feature as hardware behavior.

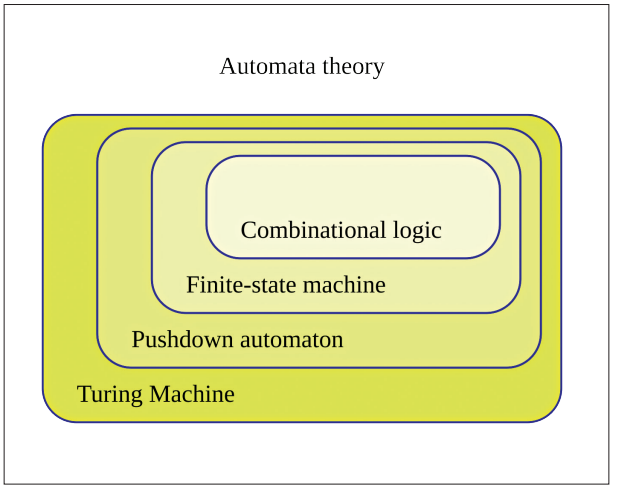
   This exact process becomes Verilog later.

Resets explained

A reset puts your system into a known starting state. Without reset, hardware can power up in random states.

Asynchronous reset

Can assert regardless of clock – useful at power-up, but introduces routing and timing implications.



Synchronous reset

Only takes effect on a clock edge – often easier to reason about in synchronous systems and FPGA flows, but it must be aligned with clock behavior.

Practical rule for beginners (especially on FPGAs):

- Prefer synchronous deassertion (reset released on a clock edge), even if assertion is asynchronous. This reduces weird half-reset states.

Reset mistakes you'll see (and how to avoid them)

1. Reset deasserted too close to clock edge → can cause inconsistent startup.
2. Reset treated like a normal signal across clock domains → can cause metastability.
3. One giant reset net going everywhere → can create routing/timing headaches (especially in large designs).

Timing concepts

You don't need to become a timing-analysis wizard yet – but you do need a basic intuition.

- Setup violations happen when data arrives too late before the sampling edge.
- Hold violations happen when data changes too soon after the edge.

This is why "it works on my desk" sometimes becomes "it fails in the field." Small variations in temperature, voltage, and routing can push a marginal design over the line.

"Try this tonight" mini labs

Lab A: Truth table → logic expression

Goal: Build a rule for a safety lock: "Motor runs only if LidClosed AND StartPressed, but EmergencyStop always forces OFF."

Step-by-step

1. Inputs: L (lid), S (start), E (emergency). Output: M (motor).
2. Write truth table for all 8 input combinations.
3. Rule: if E=1, then M=0 regardless.
4. Else M = L AND S.
5. Write expression: M = (L AND S) AND (NOT E).

That last line is exactly how you start writing combinational logic in HDL.

Lab B: Counter → PWM dimmer

Goal: Dim an LED using PWM: the LED is ON for a fraction of time, OFF for the rest.

Step-by-step

1. Use an N-bit counter that continuously increments (wraps around).
2. Choose a duty value D (0...2^N–1).
3. Output PWM = 1 when counter < D, else 0.
4. Increase D → brighter. Decrease D → dimmer.
5. Add a slow timer that changes D every few milliseconds for a fade.

This one pattern shows up in fans, backlights, motor control, audio class-D stages – everywhere.

You've just experimented with the core logic behind a million embedded systems. Congratulations!